

MULUNGUSHI UNIVERSITY

ICT 441 – ADVANCED CYBER SECURITY AND ETHICAL HACKING

LAB 1

GROUP I

MR BWALYA .M

STUDENT NUMBER	STUDENT
1. LUTUNDWA MUSONDA	202204512
2. ALBERT MUKAMBA	202300993
3. LONDENJI NGWIRA	202303830
4. TAONGA KALUBA	202303977
5. MAILESS KAMANGA	202304941
6. CHRISTOPHER NTAMBE	202301541

VULNFEED RISK ASSESSMENT AND RISK REGISTER

1.1 **RISK REGISTER**

RISK No	Title	Description	Level(1 - 5)	I (1-5)	RISK SCORE
R -02	SQL injection in search	Unsanitized ?q= parameter in GET/search allows, exposing usernames, emails, and password hashes.	5	5	25
R -01	CLOUD STORAGE MISCONFIGURATION	Vulnfeed-user-uploads GCS bucket grants AllUsers:roles/storage.objectViewer, allowing unauthenticated listing/download of all media	5	4	20
R -03	BROKEN ACCESS CONTROL ON API	GET /api/users/{user id}/posts uses the URL's user_id for the DB query but the session's user_id for display, with no match check, exposing private posts	4	4	16
R -05	MISSING MULTIFACTOR AUTHENTICATION	The/login endpoint accepts only username/password; there is no MFA flow, so a phished password grants full account takeover.	4	4	16
R -06	WEAK PASSWORD	Password validationregex /.{5,}/requires only 5 characters with no complexity rules, enabling brute-force and credential-stuffing attacks.	4	3	12
R -04	USE OF HARDCODED API KEYS	A third-party API key (e.g.. SendGrid) is hardcoded in public config.js, exposing it to every visitor's browser; it has already been abused, running up billing costs.	3	2	6

1.2 RISK REGISTER

RISK No	MITIGATION	OWNER	DUE DATE	STATUS
R -01	Remove the allUsers IAM binding immediately; set bucket to private. Serve user media via signed, time-limited URL's generated by the backend. Audit bucket access logs for the prior unauthorized downloads.	DEVOPS	48 HOURS	OPEN
R -02	Replacing string concatenation with parameterized queries / prepared statements (e.g, PDO, ORM). Add input validation and WAF rule as an immediate stopgap. Force a password rest and hash and hash rotation for all users.	BACKEND	48 HOURS	OPEN
R -03	Enforce server-side authorization: validate the session user_id against the requested resources owner before returning data. Add automated authorization tests for all object-level endpoints	Backend	1 week	Open
R -05	Implement TOTP-based MFA(e.g.. via an authenticator app) and/ or SMS/ email one time codes. Enforce mandatory MFA for admin/ staff accounts first, then roll out to all users. Add step-up authentication for sensitive actions.	security	2 weeks	Open
R -06	Enforce a minimum of 12 characters with complexity or passphrase guidance. Check newpasswords against a breached password list (e.g. HaveIBeenPwned k-anonymity API). Add rate limiting/ account lockout on repeated failed logins.	Backend	2 weeks	Open
R -04	Immediately rotate/revoke the exposed key. Move all third-party keys to backend environment variables/ a secret manager, proxying calls through the server so keys never reach the client. Add a pre-commit/CI secret-scanning check.	Frontend	1 week	Open

2. Prioritized List (1 = Highest Priority)

1. SQL Injection in Search (R-02) — Risk Score: 25
2. Cloud Storage Misconfiguration (R-01) — Risk Score: 20
3. Broken Access Control on API (R-03) — Risk Score: 16
4. Missing Multi-Factor Authentication (R-05) — Risk Score: 16
5. Weak Password Policy (R-06) — Risk Score: 12
6. Use of Hardcoded API Keys (R-04) — Risk Score: 6

3. Rationale

Highest Priority #1 — SQL Injection in Search (Risk Score: 25)

This vulnerability is ranked highest because both likelihood and impact are at their maximum. Likelihood is a 5: SQL injection via a simple UNION SELECT payload requires no authentication, no special access, and can be executed by an attacker using widely available, automated tooling (e.g. sqlmap) against a public-facing search endpoint. Impact is a 5 because a successful attack does not leak a single record — it exfiltrates the entire user table, including usernames, emails, and password hashes, which is the most sensitive dataset the application holds and directly enables downstream account takeover and credential-stuffing attacks against users on other services.

Highest Priority #2 — Cloud Storage Misconfiguration (Risk Score: 20)

This is the second-highest priority for very similar reasons to the SQL injection: the allUsers:roles/storage.objectViewer binding means literally anyone on the internet can list and download every file in the bucket with zero authentication, so likelihood is also a 5. Impact is scored one point lower, at 4 rather than 5, because the exposed data is profile pictures and uploaded media rather than credentials — a serious privacy and reputational breach, and potentially sensitive depending on the media, but it does not by itself grant an attacker the ability to log in as other users the way the leaked password hashes do.

Lowest Priority — Use of Hardcoded API Keys (Risk Score: 6)

This vulnerability is ranked lowest because, although it is a real and already-exploited issue, its blast radius is fundamentally more limited than the others. Likelihood is a moderate 3: an attacker has to actively find the key inside config.js, and while this is easy for anyone who inspects the page source, it is a narrower attack path than an unauthenticated public bucket or an injectable search field. Impact is scored a 2 because the consequence is financial abuse of a third-party billing account, not compromise of user data, authentication, or the core application/database. It should still be fixed quickly (the key is already being abused), but it does not expose customer PII or grant system access the way the top five risks do, which is why it sits at the bottom of the register.